Replication

Lab1: Bug Report Classification – ISE Coursework

Overview
These instructions outline how to fully replicate the coursework results. Since all random seeds are fixed within the code, running the script will consistently produce the identical results reported.

Code Access
The complete replication package, including the source code and pre-configured environment files, is available at: https://github.com/bmo-bmo/ISE-Bug-Classification.git.

Environmental Setup
Create a virtual environment and install dependencies:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Download the Datasets

For convenience, the CSV files are already included in the data/ folder of the repository. If you need to re-download them, they are sourced from the ISE Lab Solution repository. If the files need to be redownloaded follow the steps below

Download the five CSV dataset files from the following repository: https://github.com/ideas-labo/ise-lab-solution/tree/main/lab1

Create a folder named data/ in the project root and place all five CSV files inside it:

data/caffe.csv
data/keras.csv
data/incubator-mxnet.csv
data/pytorch.csv
data/tensorflow.csv

Verify Folder Structure
The project directory should look like this:

```
ISE-Bug-Classfication /
  main.py
  requirements.txt
  data/
    caffe.csv
    incubator-mxnet.csv
    keras.csv
    pytorch.csv
    tensorflow.csv
```

Run the Experiment
Execute the main script:

```
python3 main.py
```

The script automatically processes all five projects, with a total runtime of approximately 5 to 15 minutes depending on hardware.

Verify Results
The output prints results tables and statistical analyses for each project. Because random seeds are fixed (using random_state=i for splits and 42 for models), the output will exactly match the values reported in the coursework. GaussianNB is purely deterministic and contains no randomness.

Reproducibility Notes

- All train/test splits use random_state=i (repeat index 0 to 29).
- LinearSVC and RandomForestClassifier use random_state=42.
- GaussianNB is fully deterministic and requires no random seed.
- The TF-IDF vocabulary is learned strictly from training data in each repeat; the test set is never used during fitting.
- Results were generated using Python 3.12.3 on macOS with the versions specified in requirements.txt.